Brain-Computer Interface Simulation: Reading and Writing Information in the Brain

Vladimir Skavysh
VladimirSkavysh@cmail.carleton.ca
Carleton University
Ottawa, ON, Canada

Figure 1: The future of human-machine evolution. Credit: ChatGPT.

Abstract

Brain–computer interfaces (BCIs) enable direct interaction between neural systems and external devices, offering significant potential for assistive technologies and human–machine integration. However, the design and analysis of such systems remain challenging due to the complex, spatially distributed, and nonlinear nature of neural dynamics. In this work, we present a simulation framework for modeling neural signal propagation and electrode interaction using the Discrete Event System Specification (DEVS) formalism, specifically Cell-DEVS implemented in Cadmium v2.

The proposed model represents neural tissue as a two-dimensional grid of locally interacting cells, capturing wave-like propagation of electrical activity. Neurons are modeled as stochastic and stimulus-responsive spiking units, while electrodes are capable of both sensing (reading) and stimulating (writing) neural signals. Electrode behavior is configurable via scenario files, allowing controlled experiments with pulse timing, amplitude, and duration. Additionally, neurons can be activated not only through intrinsic stochastic firing but also through the averaged influence of neighboring potentials, enabling stimulus-induced activation.

Simulation results demonstrate the emergence of coherent propagation patterns, the interaction between electrode stimulation and neuronal activation, and the ability to distinguish between reading and writing behaviors in electrode dynamics. The framework highlights the suitability of Cell-DEVS for modeling spatially distributed neuro-electrical systems and provides a flexible platform for studying BCI-related phenomena. Limitations and future extensions, including more realistic biophysical modeling and improved propagation fidelity, are also discussed.

CCS Concepts

• **Computing methodologies** → **Modeling and simulation**; **Discrete-event simulation**; *Simulation theory*; • **Human-centered computing** → *Human computer interaction (HCI).*

Keywords

brain–computer interface, BCI simulation, DEVS formalism, discrete-event simulation, computational neuroscience, neural signal modeling, assistive neurotechnology, cyber-physical modeling

ACM Reference Format:
Vladimir Skavysh. 2026. Brain-Computer Interface Simulation: Reading and Writing Information in the Brain. In *Proceedings of Carleton University Term Projects (Term Projects).* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Brain–computer interfaces (BCIs) are systems that enable direct communication between neural activity and external devices. In

Term Projects, Ottawa, ON
2026. ACM ISBN 978-1-4503-XXXX-X/2026/04
https://doi.org/XXXXXXX.XXXXXXX

recent years, BCIs have transitioned from laboratory prototypes toward clinically viable systems capable of restoring communication and motor function in patients with paralysis [4, 6, 8]. Commercial efforts by companies such as Neuralink and Synchron further highlight the increasing feasibility and societal relevance of these technologies.

Despite rapid experimental progress, BCI systems remain difficult to design and analyze due to the complex, nonlinear, and spatially distributed nature of neural activity. Simulation provides a critical tool for understanding such systems. In particular, modeling neural signal propagation and electrode interaction enables controlled experimentation that is not feasible in vivo. Computational models can, therefore, support the development of decoding algorithms, electrode placement strategies, and stimulation protocols [1, 2].

This work focuses on the simulation of neural signal propagation and bidirectional electrode interaction using the DEVS formalism, specifically Cell-DEVS implemented in Cadmium v2. The goal is to build a spatially distributed model capable of:

- representing neurons as stochastic and stimulus-driven spiking units,
- modeling propagation of electrical activity through tissue,
- enabling electrodes to both read (detect) and write (stimulate) signals,
- analyzing resulting dynamics through simulation experiments.

Motivation. Traditional neural simulations often rely on differential equations with global coupling. In contrast, Cell-DEVS provides a local, discrete-event abstraction aligned with the physical intuition that interactions propagate through local neighborhoods.

Goals. The objective is to construct an advanced Cell-DEVS model that extends prior work by introducing:

- bidirectional electrodes (read + write),
- stimulus-induced neuron firing,
- configurable simulation scenarios,
- analysis of emergent wave propagation.

Contributions. The main contributions of this project are:

- A Cell-DEVS formulation of neural signal propagation with explicit wave dynamics,
- A novel electrode model capable of both sensing and stimulation,
- A flexible scenario-driven simulation framework,
- Empirical analysis of propagation and neuron activation behavior.

2 Background

2.1 Brain–Computer Interfaces

BCIs translate neural activity into control signals or deliver stimulation to neural tissue. Typical systems involve electrodes that measure voltage fluctuations or inject electrical current. The interaction between neural tissue and electrodes is inherently bidirectional: sensing and stimulation are tightly coupled processes [3, 5].

2.2 DEVS and Cell-DEVS

The Discrete Event System Specification (DEVS) formalism provides a mathematically rigorous framework for modeling discrete-event systems [9]. Cell-DEVS extends DEVS to spatial models where each cell interacts locally with its neighbors [7].

In Cell-DEVS:

- each cell maintains a state,
- state transitions depend on neighboring cells,
- time advances asynchronously via event scheduling.

This makes Cell-DEVS particularly suitable for modeling spatially distributed physical systems such as neural tissue.

3 Model Definition

3.1 Overview

The system is modeled as a two-dimensional Cell-DEVS lattice representing a slice of neural tissue. Each cell corresponds to a spatial location and evolves according to local interaction rules.

The cell space is defined as:

$$L = \{(i, j) \mid 0 \leq i < N_x,\ 0 \leq j < N_y\}$$

Each cell interacts with its von Neumann neighborhood (range 1).

3.2 Cell State

Each cell maintains a state:

$$S = (V, f_n, f_e, \text{is_neuron}, \text{is_electrode}, a, r, \sigma)$$

where:

- V : electrical potential (mV),
- $f_n \in \{0, 1\}$: neuron firing indicator,
- $f_e \in \{0, 1, 2\}$: electrode state (0 idle, 1 reading, 2 writing),
- is_neuron $\in \{0, 1\}$,
- is_electrode $\in \{0, 1\}$,
- a : spike age (ms),
- r : refractory time remaining (ms),
- σ : time advance.

The simulation uses a fixed resolution:

$$\Delta\tau = 0.1 \text{ ms}.$$

3.3 Neuron Model

The neuron is modeled as a stochastic spiking unit with refractory dynamics and a time-dependent membrane potential. Its firing behavior is governed by a combination of intrinsic probabilistic activation and stimulus-driven excitation arising from the aggregated electrical activity of neighboring cells.

3.3.1 Dynamics. A neuron has:

- resting potential $V_{\text{rest}} = -70$ mV,
- peak potential $V_{\text{peak}} = 30$ mV,
- spike duration ≈ 2 ms,
- refractory period T_{ref}.

The spike waveform is triangular:

$$V(a) = \begin{cases} -70 + 100a, & 0 \le a \le 1 \\ 30 - 100(a-1), & 1 < a \le 2 \\ -70, & \text{otherwise} \end{cases}$$

3.3.2 Firing Mechanisms. A neuron fires if:

(1) **Stochastic firing:** $U < p_{\text{fire}}, \quad U \sim \mathcal{U}(0,1)$
(2) **Stimulus-driven firing:**

$$\frac{1}{N} \sum_{i \in \mathcal{N}} V_i \ge \theta$$

where:
- $\mathcal{N}$ excludes the neuron itself,
- θ is configurable (default 0 mV).

3.4 Signal Propagation Model

Signal propagation is modeled as a local wave-based process in which cells emit discrete signal packets to neighboring cells. These packets propagate through the lattice via successive local interactions, with attenuation and superposition producing an approximate wave-like spread of electrical activity.

Each active cell contributes to neighboring cells via local interactions.

The potential update for a passive cell is:

$$V = -70 + \sum_k \frac{V_{0,k} + 70}{d_{\text{eff}}(r_k)}$$

where:

$$d_{\text{eff}}(r) = 1 + \frac{r-1}{10}$$

This produces:

- approximate radial decay,
- additive superposition,
- wave-like propagation.

3.5 Electrode Model

The electrode is modeled as a threshold-based sensing and stimulation unit. It detects local electrical activity when the membrane potential exceeds a configurable threshold and can inject stimulation in the form of externally defined voltage pulses with specified amplitude, duration, and timing.

3.5.1 State Modes.

- $f_e = 0$: idle,
- $f_e = 1$: reading,
- $f_e = 2$: writing.

3.5.2 Reading Behavior. The electrode detects local activity:

$$f_e = \begin{cases} 1, & V \ge \theta_e \\ 0, & \text{otherwise} \end{cases}$$

where θ_e is configurable (default −30 mV).

3.5.3 Writing Behavior. The electrode emits pulses defined by:

- amplitude V_e,
- duration T_e,
- schedule $\{t_1, t_2, \dots\}$.

During writing:

$$V = V_e \quad \text{for } t \in [t_k, t_k + T_e]$$

After the pulse:

$$V \to -70 \text{ mV}$$

This produces a rectangular waveform, contrasting with neuron spikes.

3.6 Scenario-Driven Configuration

All parameters are defined externally via JSON:

- neuron locations, probability of spiking, thresholds,
- electrode parameters (threshold, pulse timing),
- simulation duration,
- visualization settings.

This enables reproducible experimental setups without recompilation.

4 Simulation Results

Multiple simulation scenarios were executed using Cadmium v2. Each scenario explores different configurations of neurons and electrodes.

4.1 Reading Neuronal Activity

In the first simulation, a neuron is set to fire with probability $p_{fire} = 0.02$ while a nearby electrode is set to read mode throughout the simulation. The detection threshold on the electrode is set at −30.0mV, which is sufficiently low to detect the pulse from the neuron.

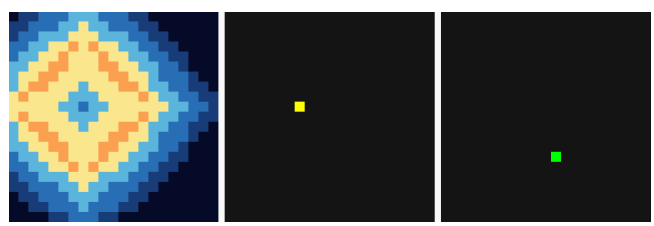

Figure 2: A neuron is firing (yellow in the 2nd window), with electric potential spreading outward (1st window). An electrode detects voltage above a set threshold value and send a signal (turns green in the 3rd window).

4.2 Writing Information onto a Neuron

In the second simulation, a neuron is set to not fire randomly ($p_{fire} = 0.0$). Instead, a nearby electrode sends 100.0mV pulses occasionally, which is sufficient to activate the neuron. The neuron fires a pulse in response to each pulse from the electrode.

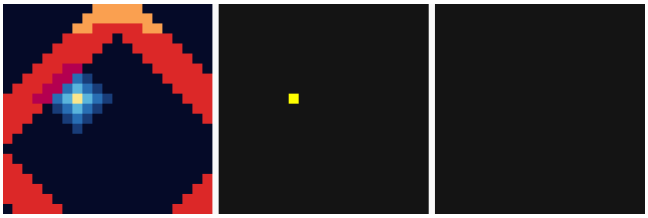

Figure 3: A pulse from an electrode activates a nearby neuron, which fires a pulse in response.

4.3 Larger Simulation with Three Neurons and Two Electrodes

The third simulation runs a scenario with multiple neurons and electrodes. With electrode thresholds and pulse amplitudes kept at default, a single pulse from an electrode can activate multiple neurons. Likewise, the electrode can detect signals from multiple neurons.

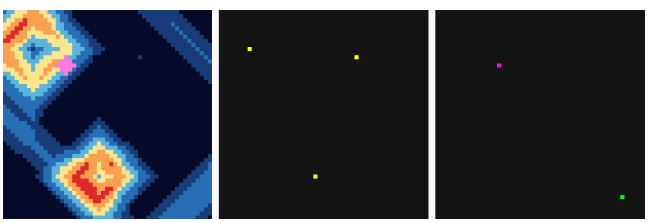

Figure 4: A larger simulation with three neuron and two electrodes shows three neurons firing, one of the electrodes sending a pulse (purple in the 3rd window) and the other electrode detecting a signal from a pervious neuronal spike.

4.4 Optimized Simulation with Three Neurons and Two Electrodes

When electrodes are input into the brain, the ideal situation is that a single electrode reads from and writes onto a single neuron. Toward this end, the amplitude of the pulses sent by the electrode needs to be adjusted so that only a single neuron gets activated. Likewise, the threshold of detection on the electrode needs to be tuned such that it reads signals from only a single neuron.

The forth simulation did just that, showing that the optimized amplitude and threshold values are $[V_{e1}, \theta_{e1}] = [60.0\text{mV}, -20.0\text{mV}]$ for electrode 1 (influencing only neuron 1) and $[V_{e2}, \theta_{e2}] = [160.0\text{mV}, -40.0\text{mV}]$ for electrode 2 (influencing neuron 3). Figures 5 and 6 show how the optimized electrode values lead to electrode 2 only interacting with the closest neuron and not with neurons farther away.

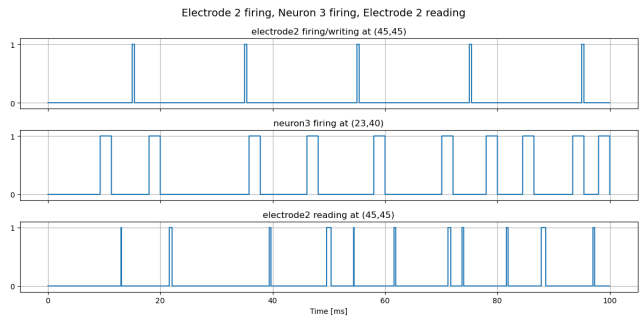

Figure 5: Pulses from electrode 2 trigger pulses from neuron 3 (the closest neuron to the electrode). In turn, pulses from neuron 3 are detected by electrode 2.

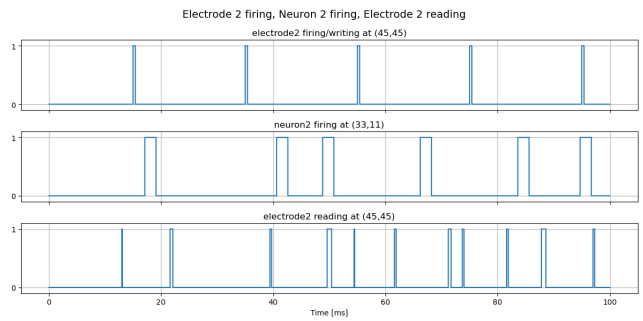

Figure 6: Pulses from electrode 2 do not trigger pulses from neuron 2 (a farther away neuron). Likewise, pulses from neuron 2 are not strongly correlated with being detected by electrode 2 (and, in fact, electrode 2 does not detect pulses from neuron 2, except when they combine with other pulses).

5 Discussion

The simulation results demonstrate that a Cell-DEVS formulation is capable of capturing several key qualitative behaviors expected in brain–computer interface systems. In particular, the model successfully reproduces (i) spatial propagation of electrical activity, (ii) stimulus-induced neuronal activation, and (iii) bidirectional interaction between electrodes and neurons.

A central observation is that complex global behavior emerges from strictly local rules. Neurons do not possess any global awareness of the system, and electrodes operate solely on local voltage measurements and pre-defined pulse schedules. Nevertheless, coordinated activity patterns arise, including wave propagation and selective neuron activation. This supports the underlying hypothesis that Cell-DEVS is well suited for modeling systems in which locality is a fundamental property of the physical process.

The electrode model further illustrates the importance of parameter tuning. In the unoptimized multi-neuron simulations, electrodes interact with multiple neurons simultaneously, both in reading and writing modes. While this behavior is consistent with the physical spread of electrical fields, it is undesirable for applications requiring precise targeting. The optimized simulations demonstrate that

appropriate choices of pulse amplitude and detection thresholds can significantly improve selectivity, enabling electrodes to interact predominantly with nearby neurons. This highlights a key practical insight: electrode design in BCI systems involves a trade-off between signal strength and spatial specificity.

At the same time, the model reveals several limitations inherent in the current formulation. The wave propagation mechanism is only approximately circular, reflecting the constraints imposed by local interactions and discrete spatial structure. As discussed in the Appendix, achieving exact geometric fidelity would require either substantially more complex state representations or modifications to the Cell-DEVS execution semantics. This limitation is not merely technical; it reflects a deeper tension between locality (a desirable modeling property) and the need to reproduce continuous physical behavior.

Another limitation is the simplified neuron model. Real neurons exhibit rich dynamics, including membrane capacitance effects, ion channel kinetics, and nonlinear threshold behavior. The current triangular waveform and binary firing state provide only a coarse approximation. However, this simplification is intentional, allowing the study to focus on interaction structure rather than detailed biophysics.

Overall, the results suggest that the Cell-DEVS approach provides a useful abstraction for studying BCI systems at an intermediate level of detail: richer than purely abstract models, but more tractable than full biophysical simulations.

6 Conclusions

This work presented a Cell-DEVS-based simulation framework for modeling brain–computer interface interactions, focusing on neural signal propagation and bidirectional electrode behavior. The model integrates stochastic neuron firing, stimulus-driven activation, and configurable electrode sensing and stimulation within a unified discrete-event framework.

The results demonstrate that:

- Local interaction rules are sufficient to generate coherent global behaviors such as wave propagation and coordinated neuron activation,
- Electrodes can be effectively modeled as both sensors and actuators within a spatially distributed system,
- Proper tuning of electrode parameters enables selective interaction with individual neurons, an important requirement for practical BCI systems.

More broadly, the study highlights the suitability of Cell-DEVS for modeling spatially distributed cyber-physical systems in which locality and asynchronous interactions are fundamental. The framework provides a flexible and extensible platform for exploring design questions that are difficult to address experimentally.

Several directions for future work remain. From a modeling perspective, incorporating more realistic neuron dynamics and improving the geometric fidelity of wave propagation would increase biological plausibility. From a systems perspective, integrating learning or adaptive mechanisms could allow electrodes to automatically tune their parameters based on observed activity. Finally, coupling the simulation with real neural data or higher-level control systems would move the model closer to practical BCI applications.

In conclusion, while simplified, the proposed model captures essential aspects of neural-electrode interaction and demonstrates how discrete-event, spatially local modeling can provide meaningful insights into the behavior of brain–computer interface systems.

References

[1] Tjeerd W. Boonstra and Michael Breakspear. 2012. Neural mechanisms of intermuscular coherence: Implications for the rectification of surface electromyography. *Journal of Neurophysiology* 107, 3 (2012), 796–807. doi:10.1152/jn.00066.2011

[2] Karl J. Friston, Andrew P. Holmes, Keith J. Worsley, Jean-Baptiste Poline, Christopher D. Frith, and Richard S. J. Frackowiak. 1994. Statistical parametric maps in functional imaging: A general linear approach. *Human Brain Mapping* 2, 4 (1994), 189–210. doi:10.1002/hbm.460020402

[3] Andrew Jackson, Jaideep Mavoori, and Eberhard E. Fetz. 2006. Long-term motor cortex plasticity induced by an electronic neural implant. *Nature* 444, 7115 (2006), 56–60. doi:10.1038/nature05226

[4] Mikhail A. Lebedev and Miguel A. L. Nicolelis. 2017. Brain–machine interfaces: From basic science to neuroprostheses and neurorehabilitation. *Physiological Reviews* 97, 2 (2017), 767–837. doi:10.1152/physrev.00027.2016

[5] Miguel A. L. Nicolelis. 2001. Actions from thoughts. *Nature* 409 (2001), 403–407. doi:10.1038/35053191

[6] Chethan Pandarinath, Paul Nuyujukian, Christine H. Blabe, Brittany L. Sorice, Jad Saab, Francis R. Willett, Leigh R. Hochberg, Krishna V. Shenoy, and Jaimie M. Henderson. 2017. High performance communication by people with paralysis using an intracortical brain–computer interface. *eLife* 6 (2017), e18554. doi:10.7554/eLife.18554

[7] Gabriel A. Wainer. 2009. *Discrete-Event Modeling and Simulation: A Practitioner's Approach.* CRC Press, Boca Raton, FL. doi:10.1201/9781420053371

[8] Jonathan R. Wolpaw, Niels Birbaumer, Dennis J. McFarland, Gert Pfurtscheller, and Theresa M. Vaughan. 2002. Brain–computer interfaces for communication and control. *Clinical Neurophysiology* 113, 6 (2002), 767–791. doi:10.1016/S1388-2457(02)00057-3

[9] Bernard P. Zeigler, Alexandre Muzy, and Ernesto Kofman. 2018. *Theory of Modeling and Simulation: Discrete Event & Iterative System Computational Foundations* (3 ed.). Academic Press, New York, NY, USA. doi:10.1016/C2016-0-03987-6

A Appendix: Circular Wave Propagation

Cell-DEVS is ideally suited for simulating systems dominated by local interactions. This is, in fact, very likely how physics actually works. Current physics thinking is that all forces are effectuated through "force carriers," which interact locally with other particles.[1] However, for macroscopic forces to appear as observed in the real world, there must be trillions upon trillions of such local interactions. For Nature, this is not a "problem" in that each particle performs a very simple "calculation." Thus, if we imagine Nature to be a highly distributed system where individual "calculations" do not need to be shared with any central "computer," then all the interactions can be easily "computed" in real time. Unfortunately, for humans attempting to simulate Nature, all the individual computations do need to be aggregated. This either makes the simulation uncomputable or means that tremendous computational simplifications need to be made somehow. Thankfully, physicists devised such simplifications by discovering physical "laws" (equations). These laws typically function "at a distance," which makes it possible to hide the true complexity of Nature. Therefore, simulations of physical systems can be done efficiently, but only if the simulations take advantage of the physical laws and—crucially—can easily implement them non-locally.

This is the challenge with using pure Cell-DEVS for simulating the circular (spherical) spread of electromagnetic brain pulses—all the interactions are local, whereas, ideally, non-local interactions could easily be programmed in into the simulation. For this reason, I have originally implemented only an approximately circular wave

[1] In the case of electromagnetic (brain) waves, the carrier particle is the photon.

propagation. In this Appendix, I describe my efforts to create a proper circular spread using pure Cell-DEVS and how such efforts have fallen short.

A.1 Intuition

In a homogeneous medium with propagation speed normalized to one, a pulse generated at the origin should reach a lattice point (m, n) after a time proportional to

$$t_{m,n} = \sqrt{m^2 + n^2}.$$

Therefore, if a cell located at (m, n) is forwarding the wave to a neighboring point (m', n'), the corresponding incremental delay should ideally be

$$\Delta t = \sqrt{m'^2 + n'^2} - \sqrt{m^2 + n^2}.$$

This suggests that a more geometrically faithful propagation rule might be obtained if local delays were chosen from this expression rather than by using a nearly uniform delay for all directions.

A.2 Initial Idea: Geometric Delay Scheduling

The first idea was to enrich the wave messages so that they would explicitly encode geometric propagation information. In particular, I considered messages carrying:

$$[V_0, m, n, m_{\text{intended}}, n_{\text{intended}}],$$

where:

- V_0 is the source neuron voltage at the moment of emission,
- (m, n) describes the current lattice displacement from the neuron,
- $(m_{\text{intended}}, n_{\text{intended}})$ identifies the intended recipient location.

The purpose of the intended-recipient fields was to ensure that a message would only be processed by the correct neighboring cell. For example, a cell at $(5, 0)$ could emit a message intended specifically for $(6, 0)$, while other neighbors receiving the same state update would inspect the message, determine that they were not the intended recipient, and ignore it.

This idea was conceptually attractive because it seemed to allow message-specific propagation timing while still working within the local Cell-DEVS framework.

A.3 Difficulty: Cell-DEVS State Exposure Semantics

However, a closer examination of the Cell-DEVS programming model revealed an important limitation. A cell computes a next state using `localComputation(...)` and then associates a *single* delay with that entire state through `outputDelay(...)`. In other words, what is delayed is the exposure of the next state as a whole, not each individual logical message contained within that state.

This had two consequences:

(1) If multiple wave messages were packed into the same cell state, they all became visible to neighbors after the same delay.
(2) There was no clean built-in mechanism for saying that one message in the state should become visible earlier and another later.

This meant that exact per-message geometric scheduling was not naturally supported by the standard Cell-DEVS cell interface.

A.4 Attempted Queue-Based and Pending-Message Designs

To work around this limitation, I experimented with versions of the model in which passive cells explicitly stored multiple pending messages in their state, together with additional timing information. The basic idea was that a cell could:

(1) receive and store several incoming wave fragments,
(2) maintain an internal list of messages still waiting to be exposed,
(3) emit them one at a time according to their computed geometric delay.

In spirit, this was similar to implementing a small event scheduler inside each cell. Although this seemed promising, in practice it led to substantial complications. In particular:

- the number of scheduled events increased rapidly,
- very small differences between message times led to many closely spaced transitions,
- the simulation showed symptoms resembling state explosion,
- unintended repeated re-processing of visible neighbor states became difficult to prevent.

Thus, although this design was closer to exact geometric propagation, it was computationally unstable and did not integrate cleanly with the Cell-DEVS semantics.

A.5 Understanding the Source of the Repeated Updates

A major conceptual clarification emerged during these experiments. In Cell-DEVS, a cell does not poll its neighbors periodically. Instead, local computation is triggered when a relevant state exposure occurs. However, when `localComputation(...)` is invoked, the cell sees the current states of *all* neighbors in its neighborhood, not only the neighbor whose state change triggered the event. A cell can, therefore, be re-invoked because of one neighbor, but during the ensuing computation it may inspect stale-but-still-visible state information from another neighbor. In the present application, this meant that old outgoing wave data could be seen again and mistakenly treated as a fresh arrival. This effect turned out to be one of the major causes of repeated unintended updates and, ultimately, event explosion.

A.6 Addition of Sequence and Neuron Identifiers

To reduce this problem, I introduced two additional identifiers into the propagated wave messages:

- `n_sequence`: an integer that increases each time a firing neuron updates its output while spiking,
- `n_neuron`: a unique integer identifier assigned to each neuron in the scenario configuration.

The corresponding message structure became

$$[V_0, m, n, m_{\text{intended}}, n_{\text{intended}}, n_{\text{sequence}}, n_{\text{neuron}}].$$

The goal was to allow each cell to distinguish:

(1) messages coming from different neurons,
(2) different successive parts of the same pulse waveform.

The neuron identifier was supplied explicitly in the JSON setup files, which made it possible to distinguish overlapping waves generated by different neurons.

A.7 Filtering Strategy

Several filtering strategies were tested. The most useful simplification was the following:

> For each neuron, keep only one message, namely a message with the largest available value of n_sequence.

If several candidate messages from the same neuron shared that largest sequence number, then only one representative was kept and the others were discarded. The purpose of this rule was not to achieve exact physical fidelity, but rather to prevent repeated re-processing of multiple logically equivalent wavefront fragments from the same neuron.

This greatly simplified the behavior of passive cells. After receiving neighboring states, the cell:

(1) adjusted the local (m, n) values according to which neighbor the state came from,
(2) rejected any message whose updated (m, n) did not match its intended recipient coordinates,
(3) grouped surviving messages by neuron identity,
(4) retained only one message with maximal sequence number for each neuron,
(5) used those remaining messages to compute the displayed local potential and to generate the next outgoing messages.

A.8 Use of the von Neumann Neighborhood

Another simplification was to switch from a Moore neighborhood to a von Neumann neighborhood. The intention was to restrict the possible directions of direct communication and thereby reduce unintended interactions through diagonal neighbors. Diagonal behavior was then reintroduced only through explicit message-generation rules in special cases.

In particular, the cells at $(\pm 1, 0)$ were given special treatment. These cells generated not only the next axial message but also two additional messages toward $(\pm 1, \pm 1)$, thereby injecting off-axis growth into the wavefront. This was intended to improve the apparent circularity of the pulse while still maintaining a simpler neighborhood structure overall.

A.9 Delay Averaging Approximation

Because a passive cell still had to expose a single next state with a single associated delay, a further approximation was introduced: if multiple surviving messages implied different geometric delays, the cell used the average of those delays as its output delay. That is, if the outgoing messages implied delays

$$\Delta t_1, \Delta t_2, \ldots, \Delta t_K,$$

then the cell used

$$\Delta t_{\text{avg}} = \frac{1}{K} \sum_{k=1}^{K} \Delta t_k.$$

This was not physically exact, but it represented a compromise between geometric fidelity and the one-state/one-delay structure imposed by Cell-DEVS.

A.10 Delay Quantization

Later experiments showed that many remaining artifacts appeared to be caused by very small numerical differences in simulated delays. These tiny differences led to many closely spaced events, which in turn amplified the update count and created visually undesirable behavior.

To reduce this effect, I quantized all computed delays to the nearest $\frac{\Delta\tau}{100}$. With $\Delta\tau = 0.1$ ms, this corresponds to a timing resolution of 0.001 ms.

This quantization reduced the number of nearly identical event times and improved numerical stability, although it did not fully eliminate the anisotropy.

A.11 Observed Outcome

These additional experiments improved the model in some respects:

- repeated re-processing of logically similar pulse fragments was reduced,
- message explosions became somewhat more manageable,
- off-axis growth could be controlled more explicitly,
- delay quantization improved numerical robustness.

However, despite all these attempts, the wave propagation achieved, tough improved, remained only approximately circular. For example, the beginning of the pulse looks close to circular (Figure 7). Yet, the ending does not (Figure 8). Thus, the overall conclusion is that perfectly circular propagation is difficult to obtain within pure Cell-DEVS formulation without introducing substantially more complicated state bookkeeping or changes to the Cell-DEVS formalism itself. Full resolution of the problem is, therefore, left as an open problem.

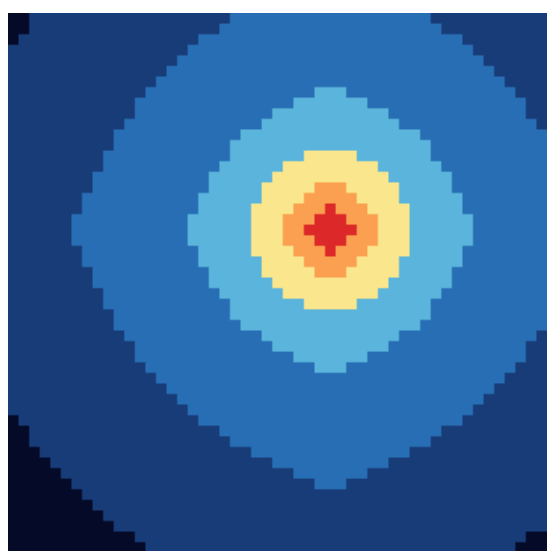

Figure 7: Beginning of an approximately circular wave.

Figure 8: Ending of an approximately circular wave, showing departure from circularity.